

AEP-X — SOA Architecture Implementation Guide

The Universal Connector Layer, designed as SOA/ESB mediation — the one part of AEP-X Ultra the microservices core doesn't fit

Source AEP-X Ultra (1).docx — 36,937-paragraph consolidated draft

Companion to ADLC-Plan, Handoff, Instructional-Manual, Microservices-Implementation-Guide

Prepared 7 July 2026

Contents

1. 1. Reconciliation — how this source relates to the existing plan
2. 2. Why SOA fits the Connector Layer, not the core
3. 3. SOA Architecture — the AEP-X Connector Bus
4. 4. Security & compliance in the SOA layer
5. 5. Build sequence addendum
6. 6. Definition of Done

1. Reconciliation — How This Source Relates to the Existing Plan

What this document is. Unlike the prior "Volume 9" outline source, this attachment is the full escalating draft chain itself — Executive Vision → PESTEL → SPARC-Native Lifecycle → Universal Plug and Connect Framework → "V3 Standards Programme" → nine further "Stage"/"Programme" re-plannings climbing to increasingly aspirational scope (Autonomous Intelligence Internet, Agent Operating System, Planetary Intelligence Grid, "AEP-X Zero — Version ∞ ") → a ~10,000-line business/governance chain (PMO, HR, BCP, ESG, ISO roadmap, M&A, IPO/Exit, 20-year Legacy Framework) → a return to concrete Sprint 0–4 build specs and the full text of RFC-0001–0010 / RA-0001–0008. It is the "3,188-page consolidated edition" the ADLC Plan's Critical Evaluation (§15.2.8) already warned about — roughly 30 restart points, each ending "the next step is...", terminology drifting between passes.

Why an SOA document at all. The source never once uses the words "SOA," "service-oriented," or "ESB" — that framing is this engagement's, not the manual's. But the source *does* independently converge, repeatedly, on a **Connector Contract** naming exactly the protocol mix classic SOA/ESB architectures exist to mediate: **Supports: MCP, REST, GraphQL, gRPC, MQTT, Kafka, NATS, SOAP, OPC-UA, Custom**, plus a generic **Connector** interface (**authenticate / discover / execute / monitor**). The **Universal Connector Layer** — Enterprise (SAP, Salesforce, MS Dynamics, Oracle, Workday, ServiceNow), Industrial (SCADA, PLC, OPC-UA, Modbus, BACnet), Productivity (Google Workspace, M365, Slack, Teams, Zoom), Developer (GitHub, GitLab, Jira, Confluence, Azure DevOps), AI Platforms (OpenAI, Anthropic, Gemini, Mistral, Ollama, vLLM, HuggingFace) — is the one part of this source not well served by the microservices style already committed. This document designs that layer as SOA and leaves the AI-native core as microservices. See [Hybrid-Architecture-and-Comparison.html](#) for why the boundary sits there.

1.1 Contradictions in this source — inherited, not re-litigated

The ADLC Plan (§15.3) already resolved the memory/cache layer and RFC-numbering contradictions in the original manual. Reading this much larger attachment end-to-end restates the same contradictions with more variants, plus two the ADLC Plan didn't have material to catch:

Topic	Conflicting statements found	Resolution adopted here
Memory layers	M0–M5 (early pass) vs M0–M6 (RFC-0005, restated ~6×)	M0–M6, RFC-0005 normative (unchanged from ADLC Plan §15.3)
Cache layers	L1–L4 (no L0) vs L0–L6 ("Seven-Level" pass) vs L0–L5 (RFC-0005, ~8×) vs concrete-TTL L0–L5 (Sprint 2.2)	L0–L5, RFC-0005 normative , TTLs from Sprint 2.2 (5 min / 1 h / 24 h / 7 d / 30 d / policy-controlled)
Identity/Trust boundary	Every Sprint 3.1–3.2 pass treats Identity and Trust as two services; the existing Microservices Guide merged them into one Trust Authority	Keep the existing merge — re-splitting now breaks already-built code for a distinction the source doesn't defend
Risk classification	Two independent 5-level taxonomies never reconciled: Safety S0–S4 (RFC-0007) vs AI Assurance AIA-R0–R4 (EU AI Act framing)	Both stay, mapped 1:1 (S0≈R0...S4≈R4) — they answer different questions (per-call vs per-system)
Platform name	"Autonomous Enterprise Protocol eXtended" (title page) vs "Agent Ecosystem Protocol Exchange" (from the investor-deck pass onward)	Cosmetic — use whichever current commercial materials use; not an architecture decision

Scope discipline check: nine of this source's ~30 passes (Stages 9–20 — Autonomous Intelligence Internet, Agent Operating System, Planetary Intelligence Grid, Singularity Framework, AEP-X Zero) describe a self-evolving, planet-scale successor to the platform being scoped in Weeks 1–12. None of that is designed here — it is exactly the "everything looks equally urgent" failure the ADLC Plan's Critical Evaluation (§15.2.1) already flagged, one order of magnitude larger. This document is scoped to the Universal Connector Layer only, buildable in the Weeks 13+ window the Microservices-Implementation-Guide left open.

1.2 Cross-walk — what's genuinely new for architecture purposes

Source concept	Status
Agent Registry, Trust Authority, Discovery, Memory, Workflow Engine, Safety Engine, Governance Engine	BUILT — unchanged
Universal Connector Layer (Enterprise/Industrial/Productivity/Developer/AI Platform/Gov-Edu)	NEW — designed in this document as SOA
Knowledge Service, ML Integration Layer, Anti-Hallucination Engine V2, Universal Cost Optimiser	NEW — designed in Microservices-Architecture-v2.html
Marketplace Engine, Certification Platform, Developer Portal	BACKLOG — Handoff \$2 deferral stands
Stages 9–20 (AOS, All, Planetary Intelligence Grid, Singularity Framework, etc.)	OUT OF SCOPE

2. Why SOA Fits the Universal Connector Layer and Not the Core

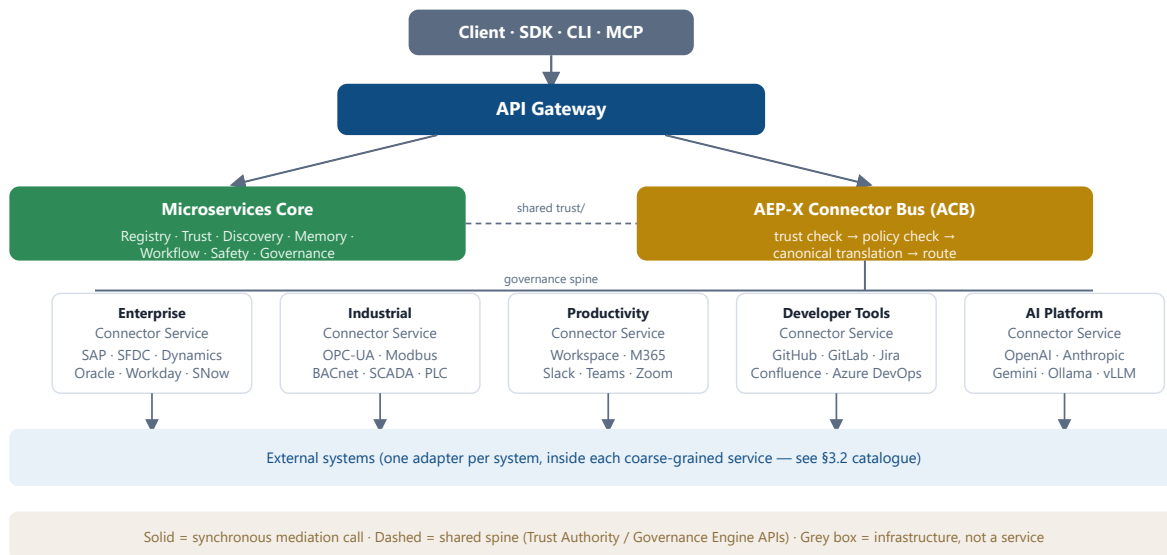
Classic SOA/ESB architecture earns its coupling and mediation overhead when a system integrates with **external systems it does not own**, speaking **heterogeneous, often legacy protocols**, changing on **vendor release cycles**, requiring **centralised governance** before any AEP-X agent touches them. That is the Connector Layer's problem, not the AI-native core's:

Property	AI-native core	Universal Connector Layer
Who owns the target system	AEP-X	External vendor / customer's estate
Protocol diversity	One (internal REST + Kafka)	Many at once: SOAP, OPC-UA, proprietary RPC, REST, GraphQL, Modbus
Release cadence	Continuous	Vendor-controlled, 6–18 months
Independent-deploy unit	One microservice	Usually a whole protocol family at once
Governance requirement	Enforced once, in Governance Engine	Enforced per external system, often per regulator (AIA-R is per-connector)
Risk if built as N microservices	N/A	N near-identical services each re-implementing auth/retry/audit — "distributed monolith of adapters"

Microservices optimise for independent deployability of cohesive capabilities you own. The Connector Layer is the opposite shape — many external protocols funnelling into one set of AEP-X-side concerns (trust check, policy check, canonical translation, audit). That's an Enterprise Service Bus's job description.

3. SOA Architecture — the AEP-X Connector Bus (ACB)

5 coarse-grained connector services · 1 bus · shared trust & governance spine



3.1 Design principles

1

Coarse-grained services, fine-grained adapters. One connector service per category (5 services), not one per external system (25+). Inside each, one **Connector** implementation per system, reusing the source's own interface verbatim:

```

class Connector:
    async def authenticate(self): pass
    async def discover(self): pass
    async def execute(self, request): pass
    async def monitor(self): pass
  
```

2

Canonical message model, no exceptions. Every connector translates its external protocol into and out of the RFC-0001 envelope before touching the bus — the same envelope already used inside the microservices core.

3

Mediation, not choreography. The core's Tier 2 talks by publishing facts to Kafka (Microservices Guide §4.2). The bus does the opposite on purpose: it centrally orchestrates trust check → policy check → translation → route → external call → response translation, because that sequencing must be enforced *before* every call to an ungoverned external system, not eventually-consistent after it.

4

Contract-first per connector. Each publishes a **Connector Contract** naming its protocols, canonical message version, required governance checks, and assigned AI risk class.

5

No connector calls another connector. Cross-connector workflows go through Workflow Engine via the Gateway — the bus stays a mediation layer, not a second orchestrator (the "smart pipes, dumb endpoints" SOA anti-pattern).

3.2 Service catalogue (SOA layer)

Connector service	External systems owned	Protocols	Governance checks
Enterprise Connector Service	SAP, Salesforce, MS Dynamics, Oracle, Workday, ServiceNow	SOAP, REST, proprietary RPC	Trust + Policy; AIA-R classified at onboarding
Industrial Connector Service	SCADA, PLC, OPC-UA, BACnet, Modbus	OPC-UA, Modbus, BACnet	Trust + Policy + mandatory Safety Engine pre-check
Productivity Connector Service	Google Workspace, M365, Slack, Teams, Zoom	REST, OAuth2-fronted APIs	Trust + Policy
Developer Tools Connector Service	GitHub, GitLab, Jira, Confluence, Azure DevOps	REST, GraphQL	Trust + Policy
AI Platform Connector Service	OpenAI, Anthropic, Gemini, Mistral, Ollama, vLLM, HuggingFace	REST (vendor-specific)	Trust + Policy + Cost Optimiser consulted before every call

Government/Education connectors are reference design only — same status as Marketplace Engine — until a pilot names a concrete system.

3.3 The bus itself

`connector-bus/` (repo root, a sibling of `services/`) — a single lightweight FastAPI mediation service plus the existing Kafka bus, **not** a commercial ESB product. `POST /bus/route` accepts an RFC-0001 envelope, checks trust and policy via the existing Trust Authority and Governance Engine APIs, forwards to the resolved connector, and publishes `connector.invoked` / `connector.failed` to the same Kafka bus Governance Engine already audits unconditionally.

4. Security & Compliance in the SOA Layer

Reuses everything already specified for the core — no new security model. OAuth2/OIDC + JWT for the calling agent; mTLS Gateway→Bus→connectors; **Trust Before Execution (Law 2)** enforced structurally at the bus, before any external system is touched; every call wrapped in the `{answer, confidence, evidence[], verification_status}` contract. AIA risk classification is assigned **once per connector at onboarding** — deliberately different from the S0–S4 safety classification, which is per call. Connectors touching Workday or any Industrial system default to AIA-R2+, requiring at minimum **Silver** assurance before going live.

5. Build Sequence Addendum (Weeks 13+)

The Connector Bus depends on Trust Authority and Governance Engine already being healthy in production — building it earlier lets ungoverned external calls through with no trust or policy check, the same failure mode the Microservices Guide flagged for early Marketplace Engine builds.

Week	Deliverable
13	Connector Bus skeleton — routing table, Kafka publish; mock connector only
14	AI Platform Connector — self-hosted/open-weights model server (Ollama or vLLM), not a commercial API — critical path for the Cost Optimiser; a metered commercial connector is added later only if needed
15	Developer Tools Connector (GitHub only)
16	Productivity Connector (Slack only)
17–18	Enterprise Connector (Salesforce only — the other five wait until this one is proven)
Back log	Industrial Connector — needs a real OPC-UA target to build against

6. Definition of Done — SOA Milestone

- `POST /bus/route` rejects a call when trust is below the connector's threshold, visible in Governance Engine's audit log within 1 second.
- At least one live connector per category except Industrial responds end-to-end through the full mediation chain.
- Every connector call is audited with `ai_risk_class` and `trust_score_at_call_time`.
- No connector service calls another connector service directly.
- Recorded in `governance/decisions/`, noting Government/Education and Industrial connectors remain reference-design-only.